Data Mining and Knowledge Discovery (2025) 39:46
https://doi.org/10.1007/s10618-025-01127-1

OLIVANDER: a counterfactual-based method to generate adversarial Windows PE malware

Luca De Rose[1] · **Giuseppina Andresini**[1,2] · **Annalisa Appice**[1,2] · **Donato Malerba**[1,2]

Received: 13 February 2025 / Accepted: 9 June 2025 / Published online: 7 July 2025

Abstract
Artificial Intelligence (AI) is transforming cybersecurity practices thanks to the amazing accuracy performance achieved with several AI-based malware detection systems. However, several recent studies have shown that AI decision models can be vulnerable to adversarial attacks. In malware detection scenarios, adversarial attacks are realistic manipulations of existing malware, which preserve the executable and malicious behaviour but evade the malware detection measures. In this study, we consider Windows Portable Executable (PE) malware, which is currently trending to prominent malware types, and we show that counterfactual explanations can be used to drive the generation of realistic adversarial Windows PE malware to evade AI-based detection. In particular, the proposed method OLIVANDER works in a black-box manner, which is the most restrictive attack option, as the evasion method interacts with the target decision system to evade by merely knowing the model input and output. The evaluation study explores the effectiveness of the proposed evasion method in terms of evasion ability, efficiency of computation, and attack transferability compared to two state-of-the-art evasion methods. In addition, the performed evaluation accounts for performances on commercial anti-malware systems.

Keywords XAI · Counterfactual explanation · Ethical adversarial learning · Windows PE malware · Evasion attack · Black-box attack · Realistic adversarial Windows PE malware

1 Introduction

Cybersecurity is the practice of defending digital systems from hostile activities. The umbrella term "malware" (MALicious softWARE) denotes various malicious files or codes that may compromise the security of digital systems by gaining unauthor-

Extended author information available on the last page of the article

ised access to digital devices, stealing data, disrupting system services and damaging digital world networks. With regard to malware detection, during the last decade, the cybersecurity field has conferred a strategic role to Artificial Intelligence (AI) as a means to help anti-malware systems to recognise malicious activities in modern anti-malware measures. Cybersecurity professionals have been able to develop AI-powered defence mechanisms that take advantage of the amazing accuracy achieved in malware detection thanks to the use of Machine Learning (ML) and Deep Learning (DL) models. However, cybersecurity opponents are also advancing at a fast rate by carrying on the battle with cybersecurity defenders. For example, in 2023, Kaspersky's detection systems registered an average of 411,000 malware every day, with an increase of nearly 3% in 2023, compared to the previous year.[1] Still, in 2023, Microsoft Windows remained the primary target for cyber-attacks, accounting for 88% of all malware-filled data detected daily. For this reason, in this paper, we focus on malware with the file format of Portable Executable (PE) in the family of Microsoft Windows operating systems, namely Windows PE malware.

The recent boom in the Windows PE malware activity has been observed despite several recent AI methods having achieved superior accuracy performance in the detection of newly emerging and previously unseen Windows PE malware (Dahiya and Chaudhary 2021). A main factor contributing to this malware boom is that a security issue commonly accompanies the use of AI decision models, which are often inherently vulnerable to adversarial attacks (Chen and Babar 2024). These are adversarial samples maliciously generated by slightly and carefully perturbing the legitimate inputs to evade the target detection model. Notably, the ethical foundations of performing "offensive" adversarial learning studies to generate adversarial samples lie in the need to help reveal the possible vulnerabilities of an AI decision-making process. Understanding AI vulnerabilities is the first step towards the "defensive" design of the best practices to protect AI systems against intentional adversaries (Choras and Wozniak 2022). In recent years, amazing results have been achieved in computer vision applications to identify and mitigate vulnerabilities of AI decision models (Akhtar et al. 2021). In Windows PE malware detection problems, a few recent studies (Demetrio et al. 2021b; Imran et al. 2024; Ling et al. 2023) have shown that adversarial Windows PE malware may be produced with catastrophic consequences for security systems of AI-powered malware detection models. However, the research to identify the vulnerabilities of these models, as well as the best practices to mitigate them, remain underexplored. Based on these premises, this work addresses the offensive goal of adversarial learning. It formulates a new evasion method to generate realistic adversarial Windows PE malware that AI developers may use to test the robustness of AI models and commercial anti-malware. Our idea is that, as in the computer vision research, developing effective offensive methods to produce adversarial Windows PE malware is the necessary springboard to pave the way for AI-powered malware detection systems designed to mantain both high accuracy on real malware and resilience to adversaries. A future direction of this study is to explore the defensive goal of adversarial learning, to enhance the robustness of AI decision

[1] https://www.kaspersky.com/about/press-releases/2023_rising-threats-cybercriminals-unleash-411000-malicious-files-daily-in-2023.

models for Windows PE malware detection by mitigating vulnerabilities disclosed with adversarial samples.

In particular, the present study uses the recently analysed similarities between popular counterfactual explanations (Guidotti 2022) and adversarial sample generation methods (Pawelczyk et al. 2022) to formulate a novel evasion method named OLIVANDER (cOunterfactuaL-based method for generatIng adVersArial wiNDows pE malwaRe). Counterfactual methods are eXplainable AI methods to provide "what-if" explanations of a decision model prediction in the form of minimal perturbation to an input sample that changes the original prediction. In OLIVANDER, counterfactual explanations are produced and interpreted to be transformed into appropriate changes applied to Windows PE codes. The applied changes aim to preserve the Windows PE-compatible format, obtain an executable file and maintain the malicious nature of the executable file. Although this study relies on counterfactuals, which is an XAI technique that has been widely explored in the recent AI literature, the novelty regards the definition of an effective methodology to use counterfactual explanations for guiding the generation of realistic adversarial Windows PE malware. To the best of our knowledge, this is the first study exploring the potential of counterfactual explanations for this offensive adversarial learning task with a focus on the challenging cybersecurity scenario of Windows PE malware. We note that OLIVANDER is a black-box evasion method since, according to the definition of black-box attack reported by Long et al. (2022), it works by knowing nothing about the AI target model (i.e., parameters, architecture and gradients) except for the model input (i.e., input data) and the model output (i.e., classification labels). Chen and Babar (2024) point out that the black-box scenario is harder to exploit than the white-box attack or grey-box attack scenarios since the latter attack scenarios assume the attackers have more information regarding the target model than the former. In particular, both white-box and grey-box adversarial attacks are developed assuming that the attackers have complete or partial knowledge of the target model, including its architecture, parameters and gradients (Long et al. 2022). In any case, the black-box scenario is a more practical assumption for attacking a general anti-malware system (Ling et al. 2023).

As a further contribution of this study, we illustrate an exploratory analysis of counterfactual insights regarding vulnerabilities leveraged to create realistic adversarial Windows PE malware. We explore the evasion ability of the proposed method compared to that of two state-of-the-art evasion methods. Finally, we analyse the transferability of the adversarial Windows PE malware generated with the considered evasion methods accounting for performances on commercial anti-malware systems.

The remainder of this paper is organised as follows. Section 2.3 introduces the background of this study in Windows PE format, LIEF for feature extraction from Windows PE files and counterfactuals. Section 3 provides an overview of recent advances to evade anti-Windows PE malware systems through the lens of adversarial learning. Section 4 presents the proposed OLIVANDER method, while Sect. 5 illustrates the results of the evaluation of the proposed method. Section 6 draws conclusions and sketches future research directions.

2 Background

2.1 Windows PE files

The PE file is the standard binary format of DLLs and Executables on the Microsoft Windows family. Its layout contains: (1) Header, (2) Section and (3) Unmapped Information areas. The Header area contains DOS Header, DOS Stub, COFF Header, PE Optional Header and Section Table. Both DOS Header and DOS Stub are used to ensure compatibility with the MS-DOS system. The COFF Header contains global information about the PE file, such as the number of sections and the creation time. The PE Optional Header is the most important header. It provides important information for the executable to be loaded by the Operative System (OS). Examples of information commonly loaded by the OS are: the file alignment that acts as a constraint on the structure of the PE file since each section of the program must start at an offset multiple to that field, the size of headers that must be a multiple of the file alignment, the offsets that point to useful structures such as the Import Table or the Export Table to find functions. The Section Table contains a list of entries that provide global information about the common sections (e.g., name, offset and size). This information allows the OS loader to find sections inside the PE file. The Section area contains a variable number of consecutive sections used to store the actual content of the executable, such as code, data and other information. Common sections contain the executable code (.text) and the global and static variables (.data). For example, additional sections contain address and size information for the import table (.idata) and export table (.edata), resource data such as icons and menus (.rsrc), uninitialised data (.bss), extended text on additional linking (.textbss), or relocation information (.reloc). The Unmapped Data Section refers to data recorded in the PE file, although they are never loaded into memory during program execution. This could include uninitialised data or debug information.

Accounting for the structure and functionality of Windows PE files, two practical byte-based manipulations are commonly used by evasion methods to preserve both the executable format and the original functionality of a Windows PE file. In particular, admitted byte-based manipulations may (1) alter bytes in suitable locations without breaking the structure or (2) inject an adversarial payload that is never executed. For example, an admitted manipulation may alter the first 58 bytes of the unused DOS header by excluding the two specific areas used to store the Magic Number and the offset to the COFF Header. Another admitted manipulation may change the name of each section by editing the corresponding section entries. Instead, an adversarial payload may be added to a Windows PE file by appending padding bytes at the end of the input binary file ("padding"), or injecting a new section in the binary file coupled with a new section entry in the Section Table ("section injection"). Both "padding" and "section injection" are manipulation operations commonly used to create realistic adversarial Windows PE malware. In fact, as illustrated by Demetrio et al. (2021b), both operations preserve the code functionality by design, as they change the byte distribution of the input code and the code structure but preserve the code execution. Specifically, the "padding" manipulation preserves the code functionality since the padding bytes are added to the end of the PE binary file without

introducing any new entry in the Section Table. Hence, the Section Table continues to map unchanged data regarding the sections of the original PE file. The PE file remains executable, and the OS loader can continue to load the common sections of the original file. On the other hand, the "section injection" manipulation adds a new section into the Section area of the PE file. The displacement caused by the injected code introduces an additional requirement to preserve the PE file functionality, that is, the need to remap the memory references of the original sections in the Section Table. This is done by adding a 40-byte entry into the Section Table to specify information regarding the size, offset and memory address of the injected section. This new entry requires recalculating offsets and memory addresses in the other entries of the Table Section to maintain the alignment. Further header information (e.g., number of sections, file alignment) is also updated in the COFF Header and PE Optional Header, respectively. As long as headers and alignments are valid, the PE binary file remains executable. The added payload is never executed with "padding" or "section injection". Hence, both manipulations do not alter the execution flow of the original file. However, the main difference between "padding" and "section injection" is that in "section injection", the memory footprint (RAM usage) during the code execution is increased since the new section is loaded into memory also if never called for the execution. Instead, in "padding", the added code is not loaded into memory.

In this paper, we describe a new evasion method that creates space to inject an adversarial payload into a Windows PE malware. The construction of the adversarial payload is driven by counterfactual knowledge, while the payload injection is done by using either "padding" or "section injection".

2.2 LIEF-based static analysis of Windows PE files

The Library to Instrument Executable Format (LIEF PE) (Thomas 2017) is a well-known, open-source library commonly used to manipulate and perform the static analysis of binary codes (e.g., Windows PE files) and extract features for AI-based decision model development. This library integrates an implementation of several binary manipulation operations, including "padding" or "section injection". In addition, it allows us to perform the static analysis of a binary PE file to represent it by a vector of 2381 raw static features extracted by parsing the binary code (Şandor et al. 2023). The extracted features are grouped into nine groups:

- The **byte histogram** (256 features) measures, for each distinct byte value, the ratio of the counts of each byte value within the file to the total number of bytes recorded in the file. For example, let us consider a Windows PE file having a size equal to 1000 Bytes and recording 200 distinct occurrences of byte 0xA. In this case, the **byte histogram** feature associated with 0xA is equal to 0.2 (computed as 200/1000).
- The **byte entropy histogram** (256 features) approximates the joint distribution $p(H, X)$ of entropy H and byte value X. This is done by computing the scalar entropy H for a fixed-length window and pairing it with each byte occurrence within the window. This is repeated as the window slides across the input bytes. LIEF uses a window size of 2048 and a step size of 1024 bytes, with 16×16 bins

that quantize entropy and the byte value. Counts are normalized to sum to unity.
- The **String** information (104 features) contains simple statistics information about printable strings. For example, it provides information about the number of strings and their average length, as well as a histogram of the printable characters present in the strings. This group also contains the number of strings associated with the file paths, URLs and registry keys.
- The **General file** (10 features) contains the file size and basic information extracted from the PE header.
- The **Header** information (62 features) contains data extracted from the COFF (e.g., timestamp, target machine) and the optional header (e.g., target subsystem, file magic) of the PE file.
- The **Section** information (255 features) describes data recorded in the section header (e.g., name, size, entropy, virtual size) having the name of the section as the entry point.
- The **Import** information (1280 features) provides information on imported functions and associated libraries extracted from the import address table, such as, for example, kernel32.dll:CreateFileMappingA.
- The **Export** information (128 features) lists information on exported functions.
- The **Data directory** information (30 features) describes values of size and virtual size of all the data directory entries recorded in the Windows PE file.

LIEF features represent the current state-of-the-art feature engineering tool used in competitive, state-of-the-art, AI-based methods for Windows PE malware detection (Barut et al. 2023; Brown et al. 2024)

2.3 Counterfactual explanations

Counterfactual methods provide explanations for model predictions by suggesting the minimal changes to input examples to modify the original prediction. According to this definition, counterfactual explanations are useful to show an actionable alternative to reverse a decision (Guidotti 2022). In cybersecurity, counterfactuals are commonly used to identify attack patterns and improve the performance of systems (Abel et al. 2020; Cotae and Reindorf 2021). In particular, the counterfactual research is closely related to adversarial learning research. Some recent studies (Freiesleben 2022; Pawelczyk et al. 2022) have explored similarities between counterfactual explanation methods and adversarial example generation methods, identifying conditions under which they are equivalent. However, Guidotti (2022) clarifies that adversarial examples are not aimed at pursuing the same goal of counterfactual explanations. In particular, adversarial examples try to fool decision models by finding a humanly imperceptible change in the input that changes predictions, while counterfactuals highlight significant features that change the predictions, with the goal of explaining the classifier. However, since counterfactual methods provide explanations in the form of minimal changes to input examples that change the model predictions, they can also be used to create adversarial attacks. For example, Kuppa and Le-Khac (2021) propose a method to create counterfactual-based poisoned attacks to compromise the confidentiality and privacy properties of AI models. However, this study

works in the feature space of network traffic data and Windows PE malware without creating realistic cybersecurity objects as we do in this paper.

In this study, we use the counterfactual explanation library DiCE (Mothilal et al. 2020) with the **byte features** extracted with LIEF. This library realises several counterfactual explainers that allow us to specify which input features to vary and their permissible ranges for valid counterfactual examples. In this study, we consider the model-agnostic method DiCE-Random that allows the generation of counterfactuals in a black-box scenario (Darias et al. 2021). In particular, it generates a set of counterfactual explanations of a sample by implementing an iterative process of random sampling from the input space (Yuan et al. 2024). Unlike the optimization-based methods implemented to generate counterfactuals, the DiCE-random method generates counterfactuals without using gradient-based techniques coherently with the black-box scenario. In particular, it produces the counterfactuals of an example by randomly perturbing some of the user-selected input features within the user-predefined bounds and querying the target decision model to check if the model's decision is changed. Counterfactuals are obtained according to a combined function to achieve the trade-off between proximity (i.e., to identify the counterfactuals that are closer to the original sample in the input feature space), sparsity (i.e., to minimize the number of features changed) and diversity (i.e., to maximize the distance among counterfactuals). Since the DiCE-Random method does not use any knowledge about the internal structure of the decision model, it may generate counterfactuals that may be less effective than counterfactuals generated performing optimisation by using the gradient, especially in high-dimensional input spaces. However, the DiCE-random method has a lower computational cost than a gradient-based method, as well as methods that use more cost expensive algorithms such as genetic algorithms or KDTree (Emiris et al. 2024). On the other hand, Yuan et al. (2024) have recently provided the empirical evidence that DiCE-Random offers a good balance between time efficiency and manifold closeness to the data innate structure.

3 Related work

Since its conception (Szegedy et al. 2014), adversarial learning has widely invested in the "offensive" scope by defining several attack methods to identify vulnerabilities in AI-based decision models (Khamaiseh et al. 2022; Zhiyi et al. 2022). The literature mainly categorises adversarial attacks into "poisoning" attacks and "evasion" attacks. Poisoning methods produce samples to contaminate a training dataset and increase the error of the learning process. Evasion methods produce samples that fool a decision model during the testing phase and are misclassified. In addition, the adversarial learning literature (Long et al. 2022) categorises attack method scenarios into "white-box", "grey-box" and "black-box" based on the attackers' knowledge of the targeted decision model. In the white-box option, attackers have a complete knowledge of the model architecture, its parameters' values, and its gradients. In the black-box option, attackers have no access to information on the target model except for the input data and the final decisions. In the "grey-box", attackers have a partial knowledge of the model. In this study, the focus is on evasion attacks in the black-box setting.

The majority of state-of-the-art evasion methods are formulated for imagery data and operate in a white-box setting. FGSM (Goodfellow et al. 2015), with its iterative variant PGD (Madry et al. 2018), is one of the most popular white-box, gradient-based, evasion methods. Although less numerous, a few evasion methods operate in a black-box mode, such as Square Attack (Andriushchenko et al. 2020) and ZOO (Chen et al. 2017). Originally formulated for imagery data, state-of-the-art evasion methods are often used with tabular data also in several cybersecurity problems. For example, Al-Essa et al. (2024) have recently used samples generated with FGSM to mitigate overfitting problems and improve the generalization and diversity of ensemble models trained in problems of Android malware detection and network intrusion detection. Macas et al. (2024) describe an extensive survey to examine the use of state-of-the-art adversarial methods in cybersecurity studies. This survey highlights that the majority of cybersecurity studies still use general-purpose evasion methods applied to a tabular representation of cyber data obtained after a feature engineering step (e.g., static analysis of a code, dynamic analysis of behaviours). So, these studies often leave aside the problem of producing realistic evasive objects in cybersecurity domains where, unlike images, there is no clear inverse mapping to the feature space (Pierazzi et al. 2020).

The generation of "realistic" adversarial malware in the problem space has gained importance in the last five years (Demetrio et al. 2021b). Focusing on Windows PE malware, which is the main topic of this study, Kreuk et al. (2018) describe the seminal evasion method, named FGSM(padding+slack) for Windows PE malware. The method is defined in a white-box mode. It is used to generate realistic adversarial Windows PE malware to fool MalConv (Raff et al. 2018)—a Convolutional Neural Network learned from raw bytes of Windows PE files. Following the same research direction, Demetrio et al. (2021b) present three white-box evasion methods, named Full DOS, Extend and Shift, to fool MalConv. Extend injects noise bytes in the DOS header, while Full DOS perturbs the bytes that are placed in the DOS header in the areas before the magic number and after the pointer to the PE header. Shift applies the shift operation to the first section to recover room to inject an adversarial byte payload. More recently, Li et al. (2023) have described another gradient-based adversarial malware generative approach, to fool MalConv. This white-box evasion method uses function-preserving manipulations to generate adversarial perturbation. It iteratively calculates the loss and backpropagation according to the output of the model, updates the embedding layer perturbation through the gradient information obtained by backpropagation, and converts the embedding layer perturbation into a byte perturbation.

On the other hand, Demetrio et al. (2021a) describe one of first black-box evasion methods, named GAMMA, for Windows PE malware detection. GAMMA uses an evolutionary algorithm to inject an adversarial payload into a Windows PE malware. It solves an optimisation problem that minimises both the evasion probability and the size of the benign injected content via a specific penalty term. The injected payload is extracted from a batch of auxiliary goodware binary files, optimising the selection and the size of benign content using selection, crossover, and mutation functions. The injection is performed with "padding" or "section injection". Demetrio et al. (2021b), as well as Imran et al. (2024) have recently conducted a systematic study of

the performance of various adversarial evasion methods showing the effectiveness of GAMMA compared to competitor white-box evasion methods. More recently, Kozák et al. (2024) have presented another black-box evasion method named AMG. It uses a reinforcement learning agent trained on a collection of auxiliary malware files to identify the optimal binary file manipulation from a set of ten functionality-preserving file manipulations (i.e., padding, section injection, append to section, rename section, break checksum, remove debug, remove certificate, increase timedatestamp, decrease timedatestamp and append new import). In particular, the agent repeatedly modifies the original malware with the optimal manipulation decided by the trained agent until the evasion of the target model.

GAMMA and AMG are evasion methods of the literature closer to OLIVANDER. All three are black-box evasion methods to produce realistic adversarial Windows PE malware. However, both GAMMA and AMG require auxiliary Windows PE files but neglect explainable insights regarding the potential vulnerabilities of the target model decisions, which may be disclosed with an XAI technique. Instead, OLIVANDER takes advantage of counterfactuals to drive the adversarial manipulation of a Windows PE malware without the need for auxiliary Windows PE files. In addition, AMG uses a sophisticated manipulation schema with a reinforcement learning agent to identify the optimal manipulation from a list of ten admitted operations. Instead, both GAMMA and OLIVANDER use a simpler manipulation schema that applies either "padding" or "section injection" according to the user input. Another difference pertains to how manipulations such as "padding" and "section injection" create the adversarial payload in the three methods. The GAMMA method uses bytes extracted from benign PE files, while AMG employs random bytes. In contrast, OLIVANDER utilizes counterfactuals to choose the bytes that fill the adversarial payloads.

4 OLIVANDER method

OLIVANDER is an offensive adversarial method formulated to attack a target AI-based decision model (target model) that is trained for Windows PE malware detection from features extracted through the static analysis of the Windows PE code performed with LIEF. Features extracted through LIEF are described in Sect. 2.2. Section 4.1 introduces motivating ideas beyond the proposed method. Section 4.2 illustrates the algorithms. Section 4.3 describes the time complexity.

4.1 Motivations

The focus of this study on exploring the vulnerabilities of an AI-based decision model that uses LIEF-extracted features is motivated by several recent studies (Barut et al. 2023; Demetrio et al. 2021b; Imran et al. 2024) showing that a decision model trained from LIEF features commonly achieves higher detection rate than a decision model trained (even with complex deep neural networks) from raw binary data. Based on this premise, let us consider a target decision model $f : LIEF \mapsto \{goodware, malware\}$ trained in the input space of LIEF for Windows PE malware detection. Let us consider any new Windows PE malware x correctly classified with f (i.e., $f(LIEF(\mathrm{x})) =$

"malware"). OLIVANDER uses knowledge disclosed with counterfactuals to identify potential vulnerabilities of f with respect to the decision $f(LIEF(\mathrm{x}))$. It leverages the counterfactual knowledge to manipulate the binary code of x and obtain a semantic-invariant, still executable, Windows PE malware x' that evades f (i.e., $f(LIEF(\mathrm{x}'))$ ="goodware"). As a code manipulation, OLIVANDER creates an adversarial payload that is injected into x to obtain x' through either a "padding" or "section injection" operation. Both "padding" and "section injection" are functionality-preserving manipulations described in Sect. 2.1.

The creation of the adversarial payload is driven by the counterfactuals of the decision $f(LIEF(\mathrm{x}))$. In OLIVANDER, counterfactuals are produced limited to the **byte histogram** features. These are LIEF features that assume values between 0 and 1, to represent the distribution of bytes in a binary file. A description of the **byte histogram** features is reported in Sect. 2.2. OLIVANDER produces counterfactuals to explain how the values of the **byte histogram** features should change from $LIEF(\mathrm{x})$ to $LIEF(\mathrm{x}')$ so that f can be tricked into misclassifying $LIEF(\mathrm{x}')$ as "goodware". Specifically, OLIVANDER generates the counterfactual of a Windows PE example allowing the **byte histogram** features of the example to change in a range varying between the values actually observed for these features in the example and 1 (that is the maximum value allowed for the **byte histogram** features).

Table 1 shows an example of a counterfactual generated for the feature vector representation extracted in the LIEF feature space for one of the Windows PE malware processed in the study dataset. These counterfactuals are produced using DiCE-Random and specifying that the **byte histogram** feature group contains the input features that may change with counterfactuals. The produced counterfactual of this example explains that the study malware may be misclassified as "goodware" whenever its binary code is modified to increase the value of the **byte histogram** features associated with byte "0xA" and byte "0x5F" from the real observed values: 0.00595 and 0.00399, to the "what-if" values: 0.39618 and 0.39449, respectively.

The rationale of generating counterfactuals for **byte histogram** features relies on the fact that the injection of an adversarial payload in a Windows PE file introduces a change in the byte distribution of the file with a specific effect on the values of the **byte histogram** features. Our intuition is that counterfactuals of **byte histogram** features can provide the guidelines to modify the byte distribution of malware to evade the target model. Accordingly, the counterfactual can be used to drive the creation of an adversarial payload so that once this payload is injected into x to obtain x', the **byte histogram** features of $LIEF(\mathrm{x}')$ roughly resemble the behaviour prescript by the counterfactual. To further support our focus on **byte histogram** features, let us anticipate that the analysis of the Mutual Info conducted in the evaluation work of

Table 1 Example of a counterfactual explanation computed for one of the Windows PE malware considered in the evaluation study

Byte histogram feature (hex value)	Original value	Counterfactual value
0xA	0.00595	0.39618
0x5F	0.00399	0.39499

This counterfactual explains that the selected malware file may be misclassified as "goodware" by replacing the real values (reported in column 2) of the features (reported in column 1) with the "what-if" corresponding values (reported in column 3)

this study (Sect. 5.2) shows that the **byte histogram** features are the group of LIEF features achieving the highest Mutual Info values computed with respect to the classification of the Windows PE files in "goodware" or "malware".

4.2 Algorithm

The pseudo-code of OLIVANDER, that is reported in Algorithm 1, includes an initialisation step and an iterative step.

Algorithm 1 OLIVANDER$(\mathrm{x}, f) \mapsto \mathrm{x}'$

```
   Data: x (binary code of a Windows PE malware), f (malware detection model)
   Result: x' (adversarial Window PE malware)
 1 begin
 2     lief_x=LIEF(x)
 3     CFmask = {}
 4     X_count = {}
 5     X'_count = {}
 6     for byte ranging between 0x0 and 0xFF do
 7         byteVal is the proportion of the occurrence of "byte" in x as it is recorded in
             lief_x
 8         CFmask.put(byte, [byteVal, 1])
 9         X_count.put(byte, byteVal*len(x)))
10     X'_count=clone(X_count)
11     CF=DiCE(lief_x, f, CFmask)
12     it=1
13     evading =false
14     while it ≤ n_max and !evading do
15         step =1
16         diverging =false
17         while step ≤ η and !diverging do
18             for byte ∈ CF.keyset() do
19                 if CF.value(byte) > normalize(X'_count.value(byte)) then
20                     X'_count.replacevalue(byte, X'_count.value(byte)+c)
21                 else
22                     X'_count.replacevalue(byte, min(X_count.value(byte),
                         X'_count.value(byte)-c))
23             step = step+1
24             if diverge(CF, X'_count) then
25                 diverging =true
26         if !diverging then
27             payload=createpayload(X'_count − X_count)
28             x'=manipulate(x, payload)
29             lief_x'=LIEF(x')
30             evading=(f(lief_x')=="malware")
31     if evading then
32         return x'
33     else
34         return x
```

In the initialisation step (Lns 2–11, Algorithm 1), OLIVANDER generates $\mathbf{lief_x}$ that is a representation of x in the LIEF input space. It produces a counterfactual

explanation *CF* of $\mathbf{lief_x}$ with respect to *f* and limited to the **byte histogram** features. Finally, it initializes two data structures—X_{count} and X'_{count}—which record the counts of each byte in the original file x and in the output manipulated file x′, respectively. The counterfactual algorithm produces the expected "what-if" changes for features of $\mathbf{lief_x}$ that belong to the group of the **byte histogram** features. For each **byte histogram** feature, the counterfactual value of the feature is produced to be greater than the actual value recorded for the given feature in $\mathbf{lief_x}$. In Algorithm 1, the permissible features and their permissible ranges are recorded in the dictionary mask *CFMask*. The counterfactual explanations are computed using DiCE-Random run to generate a counterfactual per PE file. The counterfactual explanations are recorded in the dictionary *CF*. This contains the produced counterfactual features associated with their "what-if" values. The data structure X_{count} is also built as a dictionary. For each "byte" ranging between "0x0" and "0xFF", it contains the count of the occurrences of the given "byte" in the file x. This is obtained by multiplying the **byte histogram** feature recorded for "byte" in $\mathbf{lief_x}$ by the number of bytes recorded in x. X'_{count} is initially a copy of X_{count} created to track possible changes in the byte counts to be designed according to the counterfactuals recorded in *CF*. The changes from X_{count} to X'_{count} boost the creation of the adversarial payload for the code manipulation finally done to produce the adversarial Windows PE malware x′.

In the iterative step (Lns 12–30, Algorithm 1), OLIVANDER generates a candidate adversarial payload that must be injected into x to produce a candidate adversarial file x′. The payload is created so that, once injected into x, the **byte histogram** features of the manipulated output x′ should measure values that are close to the counterfactual values produced for x. In the iterative step, the evasion capability of the candidate x′ is verified by querying *f*. The first step to constructing a candidate adversarial payload is the estimation of the payload byte distribution. To this aim, OLIVANDER performs η additions or removals of blocks of *c* bytes to the counts recorded in X'_{count}(Lns. 17–25, Algorithm 1). For each counterfactual feature, OLIVANDER verifies if the counterfactual value is greater than the normalized count of the corresponding byte recorded in X'_{count}. If the condition is verified, then OLIVANDER adds *c* bytes to the count of the byte in X'_{count}. Otherwise, it subtracts *c* bytes from the same count. *normalize*() returns the proportion of the count of the considered byte recorded in X'_{count} on the sum of counts of all bytes recorded in X'_{count}. This normalization is done to obtain the **byte histogram** information from each count value and make it comparable with the corresponding counterfactual value. In addition, OLIVANDER performs a distance divergence test to emergency stop the update of X'_{count} as soon as the Euclidean distance computed between the counterfactual values recorded in *CF* and the corresponding normalised values recorded in X'_{count} continue to increase for δ consecutive steps. In this study, we consider $\delta = 5$. As soon as the byte distribution of a candidate payload has been estimated, the candidate adversarial payload is created (Ln. 27, Algorithm 1). In particular, the payload is filled by repeating each byte whose count is changed from X_{count} to X'_{count}. For example, if X_{count}["0x0"]=175 and X'_{count}["0x0"]=225, then the payload is filled with 50 occurrences of "0x0". The adversarial payload is injected into x to produce the candidate adversarial file x′(Ln. 28, Algorithm 1). The payload injection is done by performing either "padding" or "section injection" manipulations. The default choice is "section injection". Finally,

OLIVANDER generates $\mathbf{lief}_{\mathrm{x}'}$, that is, the LIEF feature vector of x' and queries f to classify $\mathbf{lief}_{\mathrm{x}'}$(Lns. 29–30, Algorithm 1). The iterative step stops whenever it has created an adversarial candidate x' that evades the target model f or it has completed the maximum number of iterations n_{max}. The adversarial file x' whose LIEF feature vector evades the target model is outputted as adversarial Windows PE malware (Ln 32, Algorithm 1). If the iterative step stops without evading the target model, then the original Windows PE malware x is returned as output (Ln 34, Algorithm 1).

4.3 Time complexity

The time complexity of OLIVANDER mainly depends on the time spent performing both the initialisation step and the iterative step. In particular, the time complexity of the initialisation step depends on the time cost spent generating the LIEF feature vector of an input Windows PE malware and producing the counterfactual explanation of the target model decision. The time complexity of the iterative step depends on the number of iterations completed and, for each iteration, the cost spent creating an adversarial payload, generating the adversarial file candidate by performing the payload manipulation in the input binary file, generating the LIEF representation of the adversarial candidate and querying the target model. Let us consider that, in the worst case, OLIVANDER completes n_{max} iterations. Let *size* be the size of the input binary file, and $psize_{max}$ be the maximum size of an adversarial payload created in the iterative step. Let F be the cost to generate the counterfactual explanation of a decision, and Q be the cost to query the target model. According to the analysis illustrated by Wang et al. (2023), the cost F is a function of $(mn(T+k))$ where m is the number of counterfactuals generated for each sample, n is the number of features, k is the number of samples generated during evaluation, and T is the number of iterations required to optimize the loss function. The time cost of LIEF is a function of the size of the parsed binary file. The time cost of a payload creation is a function of parameters η and c. The time cost of performing a payload manipulation is a function of the payload size. Hence, the total time cost of OLIVANDER is

$$\mathrm{O}(\underbrace{size}_{LIEF} + \underbrace{F}_{CF} + \underbrace{n_{max}}_{num.\ of\ iterations}(\underbrace{\eta c + psize_{max}}_{payload} + (\underbrace{size + psize_{max}}_{LIEF} + \underbrace{Q}_{query}))).$$

The higher the number of iterations completed (and consequently the higher the number of queries performed to the target model) and the higher the total amount of bytes tentatively written with a payload manipulation to produce an adversarial candidate, the more the expected time spent to generate the output adversarial Windows PE file.

5 Evaluation study and discussion

The performance of OLIVANDER was evaluated by performing several experiments on a benchmark repository of Windows PE files. These experiments mainly aimed to explore the evasion and transferability performance of the OLIVANDER method and some related evasion methods. The implementation details of the proposed OLIVANDER method are reported in Sect. 5.1. The Windows PE file repository and

the adopted experimental set-up are presented in Sects. 5.2 and 5.3, respectively. Finally, the results are illustrated in Sect. 5.4.

5.1 Implementation details

OLIVANDER was implemented in Python 3. In the implementation used for the evaluation study, we considered a Multi Layer Perceptron (MLP) model trained for Windows PE malware detection as the target model. The input space of the MLP model was fed with the 2381 raw static features extracted by parsing the binary code of the Windows PE files with the static PE file analyser implemented in the library LIEF (version 0.9).[2] OLIVANDER also used the implementation of "padding" and "section injection" provided by the library LIEF. The MLP architecture was implemented using the high-level neural network API Keras, with TensorFlow as the back-end. It was trained with three Fully Connected (FC) layers with 512, 128 and 8 neurons and one output layer with 2 neurons, respectively. The output probabilities were obtained using the softmax activation function in the last layer. The Tanh activation function was used in all the other layers. Two Batch Normalization layers were placed before the 1st FC layer and between the 2nd and 3rd FC layers. The MLP was trained with a batch size equal to 256 and a maximum number of epochs equal to 2000. An early-stopping approach was used to stop the model earlier than the maximum number of epochs allowed if it did not improve the validation loss. All the architecture parameters described above were set as described by Connors and Sarkar (2023), who showed that this architecture outperforms several AI-based decision models in Windows PE malware detection problems. The counterfactual explanations were computed by integrating the library DiCE-Random (version 0.11),[3] used with the default parameters, but setting that the total number of counterfactuals to generate for each sample equal to 1.

5.2 Windows PE file repository

To conduct the evaluation study, we considered the Windows PE file repository, named PEMML, that was made available by Practical Security Analytics LLC.[4] We used this repository to select both the Windows PE files processed to train a target model and the Windows PE malware processed to generate the adversarial malware to fool the target model. This repository provides the binary code of 201549 labelled Windows PE files (86812 goodware, 114737 malware) with the malicious files collected from VirusShare[5] and MalShare[6] for research purposes. We used the code parser of LIEF (see details in Sect. 2.2), to perform the static analysis of all Windows PE files recorded in the PEMML repository. We also analysed the Mutual Info computed between each of the 2381 features extracted with the code static analyser of

[2] https://lief.re/.

[3] https://interpret.ml/DiCE/dice_ml.html.

[4] https://practicalsecurityanalytics.com/pe-malware-machine-learning-dataset/.

[5] https://virusshare.com.

[6] https://malshare.com/index.php.

LIEF and the label ("goodware" or "malware") of the PE file as they are obtained for each PE file of the PEMML repository. The Mutual Info analysis shows that all 256 **byte histogram** features are within the top-345 feature ranking by Mutual Info. Table 2 shows the average Mutual Info (Avg.MI) and the average Feature Rank computed according to the Mutual Info (Avg.Rank) for each group of LIEF features. These results show that the group of the **byte histogram** features contains the most relevant information for the Windows PE malware detection problem.

In the follow-up of this study, we used a training set composed of 80% of Windows PE files (i.e., 161239 PE files in total partitioned into 69450 goodware (43.07%) and 91789 malware (56.93%)) that were sampled with stratified random sampling (without replacement) from the PEMML repository to train the target model. In addition, we used the left-out 20% of Windows PE files (i.e., 40310 PE files in total partitioned into 17362 goodware (43.07%) and 22948 malware (56.93%)) to initially evaluate the accuracy performance of the target model that is trained handling a classification problem in a supervised setting. Details on the target model development are illustrated in Sect. 5.3.1. Finally, we considered a collection of 100 Windows PE malware randomly sampled from the set of testing Windows PE malware that were correctly classified as "malware" from the target model. These selected PE files were used to generate the adversarial Windows PE malware with the evaluated evasion methods. This sample set, selected for evaluating the attack ability of evasion methods, includes 51 of Trojan, 23 of Adware, 16 of Virus, 5 of Worm, 3 of HackTool, 3 of Spyware and 1 of Ransomware.

5.3 Experimental set-up

In this section, we describe the experimental set-up adopted to train the target model used in this evaluation study and the evasion methods compared.

Table 2 Average Mutual Info (Avg.MI) and Average Rank (Avg.Rank) computed on the PEMML file repository for each group of features extracted through LIEF

LIEF group	Avg.MI	Avg.Rank	LIEF group	Avg.MI	Avg. Rank
Byte histogram	**0.608**	**170.5**	Byte entropy histogram	0.365	500.9
String	0.576	236.1	General file	0.113	723.9
Header	0.060	1527.8	Section	0.027	1715.4
Import	0.006	1524.7	Export	0.012	970.5
Data directory	0.143	800.6			

The most relevant group is bold

5.3.1 Target model training and evaluation

As the target model, we considered a MLP model, learned in the supervised setting according to the description reported by Connors and Sarkar (2023), to perform Windows PE malware detection. The MLP model was learned from the training set of the PEMML repository described in Sect. 5.2. The choice of this MLP model as the target model of the evaluation work was based on the results of the evaluation illustrated by Connors and Sarkar (2023). Their study shows that such MLP model achieves higher detection accuracy than several AI-based malware detection methods analysed. As confirmation, we verified that the MLP model trained on the training set of the PEMML repository achieved F1=99.02 (with Precision=99.13 and Recall=98.91) on the testing set of the same repository. The high accuracy performance of the MLP model legitimates our decision to use such MLP model as the target model of the evasion methods evaluated in this study. In fact, the task of identifying the vulnerabilities that may allow an ethical hacker to evade such an accurate target model can actually be considered a tricky attacking task.

5.3.2 Compared evasion methods

We compared the performance of OLIVANDER and its competitors: AMG (Kozák et al. 2024) and GAMMA (Demetrio et al. 2021a). Both AMG and GAMMA are state-of-the-art evasion methods described in the recent ethical hacking literature to generate adversarial Windows PE malware to evade an AI-based target model in a black-box scenario. The Python implementation of AMG and GAMMA was made publicly available by the authors to perform experiments with these evasion methods for scientific scopes. Similarly to OLIVANDER, the implementation of GAMMA uses "padding" or "section injection" manipulations provided by the LIEF library. In particular, both OLIVANDER and GAMMA were implemented to manipulate a PE file by using "padding" or "section injection" through a counterfactual-based algorithm and a genetic algorithm, respectively. In the follow-up of this study, we denote $\text{OLIVANDER}^{\text{PAD}}$ and $\text{GAMMA}^{\text{PAD}}$ the configurations that use "padding", $\text{OLIVANDER}^{\text{INJ}}$ and $\text{GAMMA}^{\text{INJ}}$ the configurations that use "section injection", respectively. The original formulation of AMG, that was used in this evaluation study, was implemented to integrate a reinforcement learning agent to decide the adversarial manipulation operations to apply from a set of ten admitted operations, namely padding, section injection, append to section, rename section, break checksum, remove debug, remove certificate, increase timedatestamp, decrease timedatestamp and append new import. The agent was trained as described by Kozák et al. (2024) with Windows PE malware sampled from the same training set used in this study to learn the target model. The implementation available for AMG uses the implementation of these manipulations provided by the PEFile library.[7]. As both OLIVANDER and GAMMA can use only one type of manipulation operations per execution (either "padding" or "section injection"), for a safe comparison, we run two additional configurations of AMG, denoted AMG^{PAD} and AMG^{INJ}, that use

[7] https://github.com/erocarrera/pefile

"padding" and "section injection", respectively. Finally, we recall that both AMG and GAMMA were formulated to use auxiliary binary files to produce the adversarial version of a Windows PE malware. In this study, we used the training set already considered to train the target model as the source of the auxiliary binary PE files that GAMMA and AMG required.

The compared evasion methods operate in a black-box attacking scenario, as they only need to know the input and output of the target model that is repeatedly queried as an oracle. Hence, the comparison of their performance is safe. In addition, similarly to OLIVANDER, the authors of both AMG and GAMMA designed and experimented with their methods within the feature space produced with LIEF. Finally, all the tested configurations of both OLIVANDER and GAMMA generated each adversarial Windows PE malware by repeatedly manipulating the binary code of the original file, stopping the iterative process as the malware generated evades the target model or the maximum number of iterations has been achieved. For a safe comparison, the configurations of OLIVANDER and GAMMA were all tested with the maximum number of iterations n_{max} set equal to 1000. This set-up was not detrimental to the GAMMA's evasion ability, which was originally tested by the authors completing 100 iterations only. The remaining input parameters of the related evasion methods were set as suggested by the methods' authors.

5.4 Results and discussion

The main goals of the experimental study are:

1. To analyse the counterfactuals generated with OLIVANDER and examine the potential vulnerabilities they disclose in the input space of the target model (Sect. 5.4.1).
2. To explore the effect of input parameters η and c on the evasion ability and the computation performance (computation time, number of queries, amount of bytes) of the two configurations of OLIVANDER realised for this study: OLIVANDER $^{\mathrm{PAD}}$ and OLIVANDER$^{\mathrm{INJ}}$(Sect. 5.4.2).
3. To compare the performance of OLIVANDER–configurations: OLIVANDER $^{\mathrm{PAD}}$ and OLIVANDER$^{\mathrm{INJ}}$–with that of the two evasion methods AMG—configurations: AMG, AMG$^{\mathrm{PAD}}$ and AMG$^{\mathrm{INJ}}$ and GAMMA—configurations: GAMMA$^{\mathrm{PAD}}$ and GAMMA$^{\mathrm{INJ}}$(Sect. 5.4.3).
4. To evaluate how the adversarial Windows PE malware generated in this evaluation study transfer to another state-of-the-art AI-based malware detection model, as well as to real-world, commercial anti-malware systems (Sect. 5.4.4).
5. To evaluate how the adversarial Windows PE malware generated in this evaluation study are able to evade a potentially less vulnerable version of the target model trained with an Adversarial Training approach (Sect. 5.4.5).

Experiments were conducted on a Windows 11 machine with an Intel® i7-12700 H CPU, a GPU GeForce RTX 3080TI and 64 GB of RAM to perform the evaluation study of this work. We used a Ubuntu 22.04 virtual machine to run the malicious code. All the evasion methods considered in this evaluation study apply code manip-

ulations designed to preserve the code functionality by design as they were implemented in the LIEF library (for OLIVANDER and GAMMA) and PEFile library (for AMG). Accordingly, in the experiments, we checked that all the adversarial Windows PE malware generated with both OLIVANDER and the related evasion methods can be really executed in the online sandboxes used by Virus Total.[8] This test provided further evidence that the implementation of the different binary manipulation operations provided by both the LIEF library and the PEFile library and used by the compared evasion methods actually preserve the code functionality by design as expected from the definition of these operations. Finally, to examine the effectiveness of our decision of integrating DiCE-random in OLIVANDER, we conducted a further experiment to compare the performance of DiCE-random to that of DiCE-gradient. Appendix 7 illustrates the results of this comparative study.

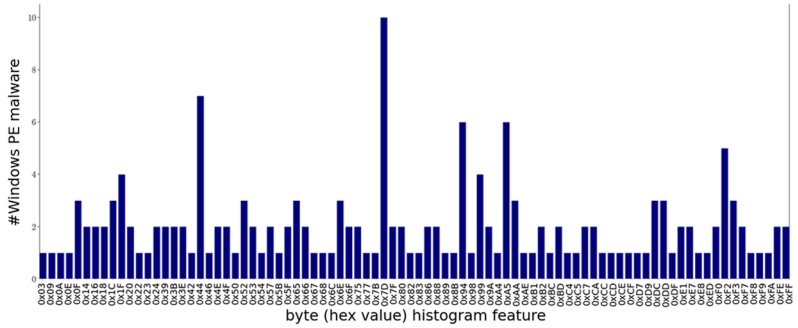
5.4.1 Counterfactual explanations analysis

In this section, we analyse how the counterfactuals generated for the Windows PE malware of this study explain some of the potential vulnerabilities of the input **byte histogram** features in the queried target decision model. These are the input features of the target model examined with the counterfactuals to drive the generation of the adversarial Windows PE malware with OLIVANDER. The counterfactuals generated for the study dataset changed 86 out of 256 **byte histogram** features. This highlights that some of the vulnerabilities of the considered target decision model can be related to clear changes in the distributions of specific bytes in the binary file.

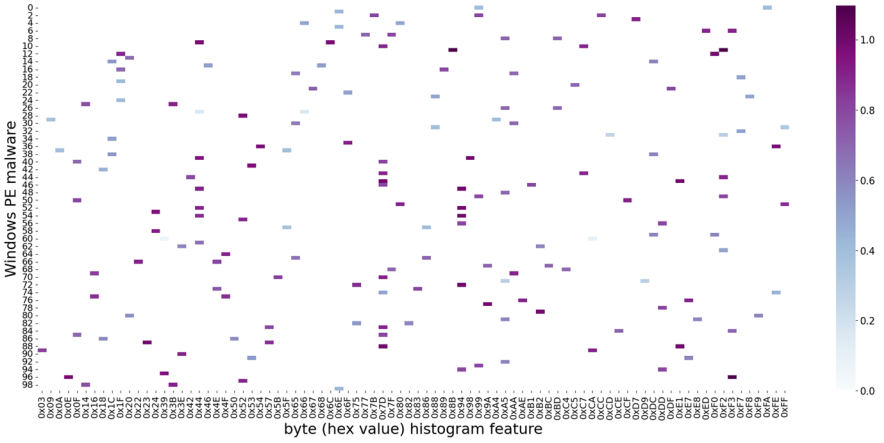
Figure 1a reports the histogram of the number of Windows PE malware for which each **byte histogram** feature was selected to generate a counterfactual. This plot shows that the **byte histogram** feature of byte "0x7D" is one of the most frequently input features of the target decision model changed to create counterfactuals and, consequently, considered to drive the creation of an adversarial Windows PE malware generated with OLIVANDER. In addition, the **byte histogram** feature of byte "0x44" is the runner-up of this histogram plot. Hence, these counterfactual insights reveal the information that the addition of adversarial payloads filled with these two bytes often provides the opportunity to evade the target model.

Focusing the attention on the **byte histogram** features that were involved in the counterfactuals produced in the study, Fig. 1b shows the heatmap of the differences computed between the values observed for each selected **byte histogram** feature in both the original Windows PE malware and the corresponding counterfactual generated. The heatmap of these counterfactual-guided changes explains that the **byte histogram** feature of byte "0x7D" should be significantly increased (with one exception) to achieve a change in the model decision, while the **byte histogram** feature of bytes "0x1F", "0x5F" and "0x68" needs smaller changes with respect to the original values to evade the model decision.

[8] https://www.virustotal.com.



(a) Histogram of the number of Windows PE malware (axis Y) for which each **byte histogram** feature (axis X) was selected for creating a counterfactual



(b) Heatmap of the differences computed between the values measured for each **byte histogram** feature in both the original Windows PE malware and the counterfactual generated. Differences are shown for each **byte histogram** feature (axis X) and for each Windows PE malware (axis Y) in this study.

Fig. 1 Analysis of the **byte histogram** features selected to create counterfactual explanations: number of Windows PE malware in which the input feature was changed with a counterfactual (**a**), and difference between the values of the feature observed in both the original Windows PE malware and the generated counterfactual (**b**). Both plots show the **byte histogram** features (with the hex notation) for which at least one counterfactual was generated in the study dataset

5.4.2 Sensitivity analysis

In this section, we illustrate the results of the sensitivity study performed to analyse the effect of the input parameters: η(number of steps performed within an iteration to create the payload of the manipulation) and c (number of bytes potentially added or removed, for each counterfactual feature, in each step of a single iteration), on the evasion performance, as well as on the computation time consumed, the number of queries performed, and the size of adversarial payloads tentatively written by both OLIVANDER$^{\text{PAD}}$ and OLIVANDER$^{\text{INJ}}$. We considered $\eta = 1000$ and $c = 100$ as the default parameter configuration for the two versions of OLIVANDER. We performed experiments to explore the performance of OLIVANDER$^{\text{PAD}}$ and

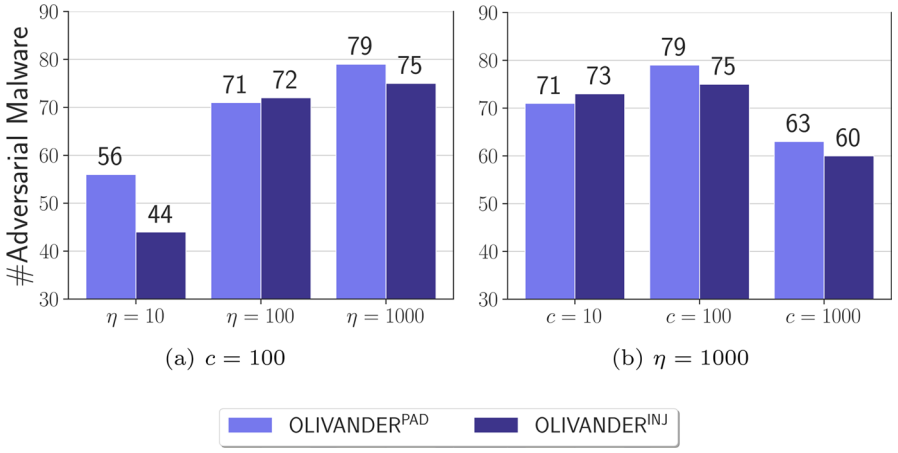


Fig. 2 #Adversarial Malware of OLIVANDER$^{\text{PAD}}$ and OLIVANDER$^{\text{INJ}}$ with $c = 100$ and η varying among 10, 100 and 1000 (**a**), and $\eta = 1000$ and c varying among 10, 100 and 1000 (**b**)

OLIVANDER$^{\text{INJ}}$ by setting $c = 100$ and varying η among 10, 100 and 1000, as well as setting $\eta = 1000$ and varying c among 10, 100 and 1000.

5.4.2.1 Evasion ability Figure 2a and b show the #Adversarial Malware – the number of adversarial Windows PE malware that evaded the target model being wrongly classified as "goodware". This metric was measured in all the tested configurations of OLIVANDER$^{\text{PAD}}$ and OLIVANDER$^{\text{INJ}}$. As the adversarial malware generation was performed using the same test set of 100 Windows PE malware for each evasion method, the highest the value of #Adversarial Malware, the more the ability of the evasion method to fool the target model. Figure 2a shows that #Adversarial Malware increases as the number of steps η increases. Both OLIVANDER$^{\text{PAD}}$ and OLIVANDER$^{\text{INJ}}$ achieve the highest value of #Adversarial Malware when the two methods were run with $\eta = 1000$. We also note that the evasion ability of both OLIVANDER $^{\text{PAD}}$ and OLIVANDER$^{\text{INJ}}$ is similar when the two methods were run with $\eta = 100$ and $\eta = 1000$, while the two methods differ significantly from each other when run with $\eta = 10$. On the other hand, Fig. 2b shows that OLIVANDER$^{\text{PAD}}$ and OLIVANDER$^{\text{INJ}}$ achieve the highest value of #Adversarial Malware with $c = 100$, while both methods achieve the lowest value of #Adversarial Malware with $c = 1000$. The considerations reported above are independent of the manipulation used. In conclusion, the highest evasion performance is obtained with OLIVANDER$^{\text{PAD}}$ run with $\eta = 1000$ and $c = 100$, while the runner-up is OLIVANDER$^{\text{INJ}}$ also run with $\eta = 1000$ and $c = 100$.

5.4.2.2 Computation time, number of queries, and payload size Figure 3a–d show the radar charts of: the computation time (Time) spent in average processing every study Windows PE malware, the number of queries (♯Queries) performed in average to the target model and the number of bytes (♯Bytes) tentatively written in average. Figure

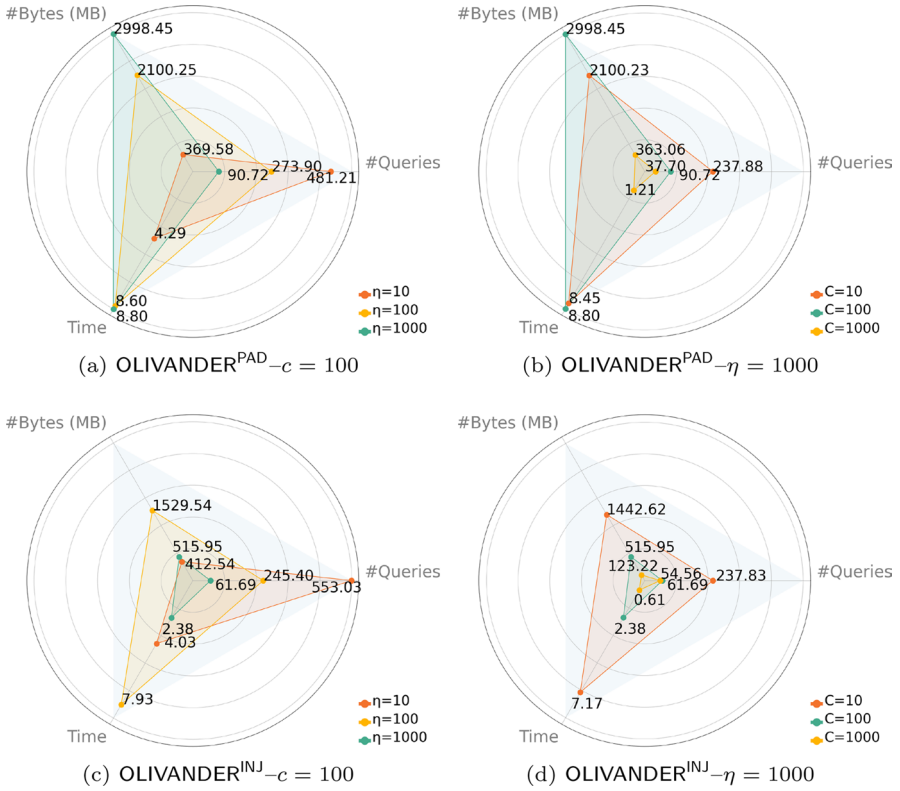


Fig. 3 Average number of queries (♯Queries) to the target model, average amount of bytes (♯Bytes) and average computation time (Time in minutes) spent from OLIVANDER$^{\text{PAD}}$ and OLIVANDER$^{\text{INJ}}$ processing every Windows PE malware of the study dataset with $c = 100$ and η varying among 10, 100 and 1000, and $\eta = 1000$ and c varying among 10, 100 and 1000

3a and b refer to OLIVANDER$^{\text{PAD}}$, while Fig. 3c and d refer to OLIVANDER$^{\text{INJ}}$. These radar charts show that, as either η or c increase, ♯Queries decreases in both methods. However, these charts do not reveal any general pattern to describe a specific trend in the effect of η and c on ♯Bytes. This depends on the fact that, in each of the η steps of an iteration, OLIVANDER chooses between two operations, that is, "adding c bytes" or "removing c bytes" from the candidate adversarial payload. This choice is made solely on the basis of the outcome of a counterfactual-driven test (Algorithm 1, lines 19–22). As this counterfactual-dependent choice introduces variability in the size of candidate payloads, ♯Bytes turns out to be ultimately independent of η and c. On the other hand, the size of the adversarial payload tentatively written with a code manipulation appears to be the dominant factor influencing the computation time. In general, the radar chart analysis shows that the higher the value of ♯Bytes, the more the Time spent completing the process to produce an adversarial Windows PE malware. In addition, it shows that an increase in the value of ♯Queries does not necessarily lead to an increase in the Time unless also accompanied by an increase in– ♯Bytes. Finally, we note that the default parameter configuration

Table 3 # Adversarial Malware and average computation Time (spent in minutes) of AMG, AMG^{PAD}, AMG^{INJ}, $\text{GAMMA}^{\text{PAD}}$, $\text{GAMMA}^{\text{INJ}}$, $\text{OLIVANDER}^{\text{PAD}}$ and $\text{OLIVANDER}^{\text{INJ}}$

Method	# Adversarial Malware	Time (mins)
AMG	35	5.02
AMG^{PAD}	32	1.43
AMG^{INJ}	6	1.16
$\text{GAMMA}^{\text{PAD}}$	10	5.48
$\text{GAMMA}^{\text{INJ}}$	58	0.73
$\text{OLIVANDER}^{\text{PAD}}$	79	8.80
$\text{OLIVANDER}^{\text{INJ}}$	75	2.38

($\eta = 1000$ and $c = 100$) of $\text{OLIVANDER}^{\text{PAD}}$ spends more computation time than the default configuration of $\text{OLIVANDER}^{\text{INJ}}$ since it performs a higher number of queries (90.7 vs 61.7) and writes a higher total amount of bytes on average (2998.4 vs 1529.5).

5.4.3 Competitor analysis

Table 3 reports the #Adversarial Malware and the average Time (spent in minutes) of AMG, AMG^{PAD}, AMG^{INJ}, $\text{GAMMA}^{\text{PAD}}$, $\text{GAMMA}^{\text{INJ}}$, $\text{OLIVANDER}^{\text{PAD}}$ and $\text{OLIVANDER}^{\text{INJ}}$. The results show that $\text{OLIVANDER}^{\text{PAD}}$ produces the highest number of adversarial Windows PE malware that fool the target model in this comparative study, with $\text{OLIVANDER}^{\text{INJ}}$ as runner-up. However, the higher evasion performance of $\text{OLIVANDER}^{\text{PAD}}$ is achieved at the cost of the higher computation time spent in average processing each Windows PE malware. In general, the evasion methods that use the padding manipulation ($\text{OLIVANDER}^{\text{PAD}}$, AMG^{PAD} and $\text{GAMMA}^{\text{PAD}}$) spent more computation time than their counterparts ($\text{OLIVANDER}^{\text{INJ}}$, AMG^{INJ} and $\text{GAMMA}^{\text{INJ}}$) that use the section injection manipulation. In general, $\text{OLIVANDER}^{\text{INJ}}$ achieves a good trade-off between evasion ability and computation time.

Finally, with regard to the two configurations of OLIVANDER, we report that the average computation time spent on generating the counterfactual of each Windows PE malware is 1.93 min. This quantifies the impact of the counterfactual cost on the total cost spent by $\text{OLIVANDER}^{\text{PAD}}$(8.80 min) and $\text{OLIVANDER}^{\text{INJ}}$(2.38 min), respectively. As counterfactuals were computed before manipulating Windows PE files, this cost is independent of the decision to use “padding” or “section injection”.

5.4.4 Transferability analysis

The term transferability refers to the property of an adversarial sample that was created to fool a target model to also be able to fool other decision models (Gu et al. 2024). To evaluate the transferability of the adversarial Windows PE malware produced with the evasion methods considered in this study, we analyse how many adversarial malware created to fool the MLP target model are also able to evade the MalConv model (Raff et al. 2018) and the IForest model (Liu et al. 2008). MalConv is an end-to-end deep neural model that takes the raw bytes of a PE file as input and predicts the mali-

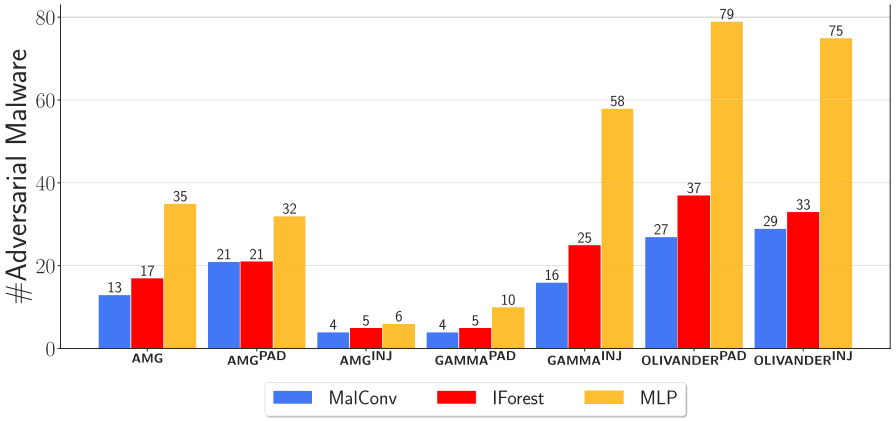


Fig. 4 Number of adversarial malware (♯Adversarial Malware) produced with AMG, AMG^{PAD}, $\text{AMG}^{\text{INJ}}\text{GAMMA}^{\text{PAD}}$, $\text{GAMMA}^{\text{INJ}}$, $\text{OLIVANDER}^{\text{PAD}}$ and $\text{OLIVANDER}^{\text{INJ}}$ that also fool IForest and MalConv models on the original number of adversarial Windows PE malware created to evade the MLP target model

cious behaviour of the input file. This is commonly considered a state-of-art model for Windows PE malware detection (Demetrio et al. 2021b; Imran et al. 2024). The architecture is trained leveraging the supervision of the training set labels. MalConv architecture includes an embedding layer, a convolution layer, and an FC layer for the final softmax classification. In this study, we used the pre-trained the MalConv model[9] described by Anderson and Roth (2018). This model was trained using 1 Mb bytes of input Windows PE files. Specifically, the model was trained by appending zeros to the end of PE files smaller than 1 Mb and removing the last bytes of PE files greater than 1 Mb. MalConv model is also used as the target model of several white-box evasion methods such as evasion methods (e.g., Extend, FullDos, Shift and FGSM padding+slack) described and experimented by Demetrio et al. (2021b). In addition, MalConv model has been recently used in the experimental comparison of the performance of various evasion methods illustrated by Imran et al. (2024). On the other hand, IForest is an unsupervised anomaly detection method that has often been used in various malware detection tasks (Pawar et al. 2024; Shi et al. 2024). It is particularly effective for identifying outliers, so it may be useful for better detecting adversarial malware that may not exhibit patterns familiar to a supervised model. An IForest uses a collection of decision trees to isolate outliers by randomly selecting a feature and then randomly selecting a split value between the maximum and minimum values of the selected feature. In this study, we considered the IForest used with the LIEF feature vector. Figure 4 shows the number of adversarial Windows PE malware, produced with AMG, AMG^{PAD}, AMG^{INJ}, $\text{GAMMA}^{\text{PAD}}$, $\text{GAMMA}^{\text{INJ}}$, $\text{OLIVANDER}^{\text{PAD}}$ and $\text{OLIVANDER}^{\text{INJ}}$, which are also able to fool the MalConv model (blue coloured bars) and the IForest model (red coloured bars) on the original number of adversarial malware created to evade the MLP target model (yellow coloured bars). The results show that the two configurations of OLIVANDER pro-

[9] https://github.com/endgameinc/malware_evasion_competition.

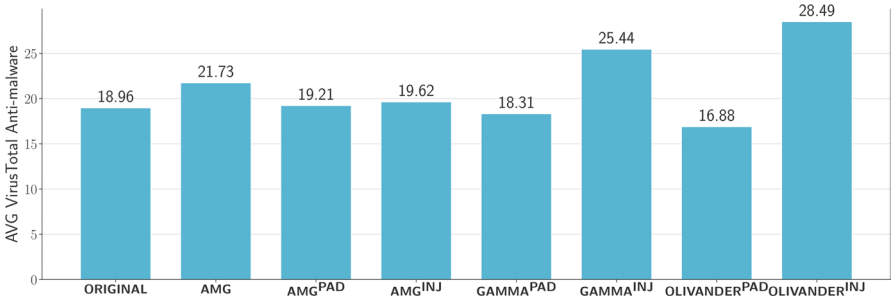


Fig. 5 Average (AVG) number of real-world anti-malware evaded in VirusTotal by the original malware set (ORIGINAL) and the malware set modified by replacing original malware with adversarial malware produced with AMG, AMG^{PAD}, AMG^{INJ}, $\text{GAMMA}^{\text{PAD}}$, $\text{GAMMA}^{\text{INJ}}$, $\text{OLIVANDER}^{\text{PAD}}$ and $\text{OLIVANDER}^{\text{INJ}}$

duces the highest number of adversarial malware that evade both the MalConv model and the IForest model in addition to the MLP target model.

To complete this transferability study, we explore the ability of the adversarial Windows PE malware generated from the compared evasion methods to evade the 77 real-world anti-malware engines accessed via VirusTotal between June 2024 and February 2025. Figure 5 shows the average number of anti-malware systems evaded in VirusTotal by each Windows PE malware from the original malware dataset and the adversarial malware dataset. We recall that the considered evasion methods were not able to generate adversarial Windows PE malware for some of the tested input PE malware. In these cases, all the evasion methods were implemented to return the input PE malware. So, for a safe comparison, for each evasion method, we submitted the list of 100 PE files produced by the evasion method to VirusTotal, being aware that each list may contain the input PE malware in the absence of an adversarial Windows PE malware generated by the considered evasion method.

This plot shows that all evasion methods, except for $\text{GAMMA}^{\text{PAD}}$ and $\text{OLIVANDER}^{\text{PAD}}$ increase, on average, the number of real-world anti-malware evaded thanks to the adversarial manipulations used. The results achieved with the "padding" manipulation are consistent with the conclusions recently drawn by Louthánová et al. (2024), who have shown that GAMMA achieves worse evasion performance using "padding" in place of "section injection" by failing to mislead commercial anti-malware programs. In general, the VirusTotal results show that $\text{OLIVANDER}^{\text{INJ}}$ achieves, on average, the higher evasion ability versus commercial anti-malware systems, with $\text{GAMMA}^{\text{INJ}}$ as runner-up. These results shed new light on the performance of $\text{OLIVANDER}^{\text{PAD}}$, which, although capable of generating a high number of adversarial malware that can evade the target model or even another AI model, is not capable of transferring the same to commercial anti-malware systems. Instead, $\text{OLIVANDER}^{\text{INJ}}$ achieves the best trade-off in terms of evasion and transferability ability also versus commercial anti-malware systems.

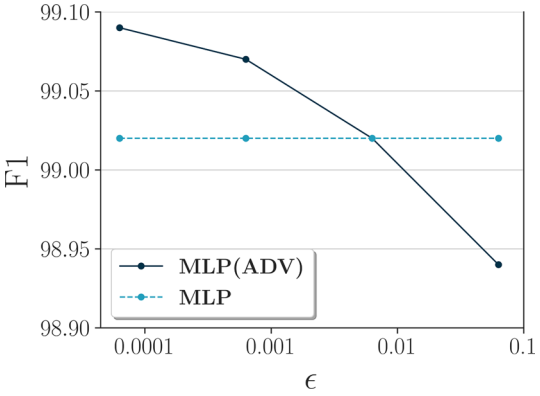


Fig. 6 F1 score of the MLP(ADV) model trained with the Adversarial Training approach by considering the training set of the PEMML repository augmented with adversarial training samples generated with FGSM run by varying ϵ among 0.0001, 0.001, 0.01 and 0.1. The F1 score was measured on the original testing set of the PEMML repository. The dot line denotes the F1 score of the original MLP model trained without Adversarial Training

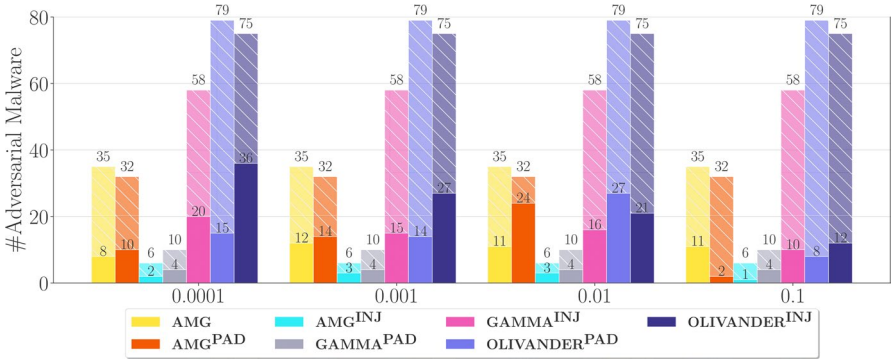


Fig. 7 Number of adversarial malware (♯adversarial Malware) that evade the MLP(ADV) models of Fig. 6 on the original number of adversarial Windows PE malware originally produced with AMG, AMG^{PAD}, AMG^{INJ}, $\text{GAMMA}^{\text{PAD}}$, $\text{GAMMA}^{\text{INJ}}$, $\text{OLIVANDER}^{\text{PAD}}$ and $\text{OLIVANDER}^{\text{INJ}}$

5.4.5 Adversarial training

The adversarial training strategy is originally illustrated by Szegedy et al. (2014) as a mechanism to make decision models less vulnerable to adversarial attacks by training models from clean samples augmented with adversarial samples. Adversarial Training has recently proved effective in several recent cybersecurity studies as a means to mitigate the overfitting risk and gain accuracy on clear data (Al-Essa et al. 2024), as well as to improve the robustness of decision models to evasion samples (Imran et al. 2024) at the testing time. Hence, to explore the offensive ability of the evasion methods of this study, we also analyse the number of adversarial malware generated by each evasion method that continue to fool the target MLP model re-trained with adversarial training. We performed the adversarial training using adversarial samples generated at the training time with the Fast Gradient Sign Method (FGSM). This method was selected as it outperformed several adversarial sample generation techniques for adversarial training in the study of Al-Essa et al. (2024). Figure 6 shows the F1 score of the original MLP model (the one used as the target model in the evasion methods) and the new MLP(ADV) models trained from the training set

of PEMML repository performing Adversarial Training with FGSM run varying the noise perturbation among 0.0001, 0.001, 0.01 and 0.1. The F1 score was measured on the clear, original test set of the PEMML repository. Notably, the MLP(ADV) model outperforms (or performs equally to) the original MLP model until $\epsilon < 0.1$). As an ideal decision model should be accurate on clear data and robust to adversarial samples, we focus the following analysis on MLP(ADV) models trained with $\epsilon \leq 0.1$. Figure 7 shows the number of adversarial Windows PE malware generated from each evasion method that continues fooling the less vulnerable counterpart MLP(ADV) model trained with Adversarial Training. The offensive ability of an evasion method, in general, tends to decrease as ϵ increases. The only exception is observed with AMG^{PAD} where the number of adversarial Windows PE malware that fool the MLP(ADV) model increases as ϵ increases until 0.01, while it drastically decreases with ϵ equal to 0.1. Finally, we observe that $\text{OLIVANDER}^{\text{INJ}}$ produces, in general, a higher number of adversarial malware that also fool the MLP(ADV) models, except for the MLP(ADV) with ϵ equal to 0.01 for which the most offensive method appears $\text{OLIVANDER}^{\text{PAD}}$ with AMG^{PAD} as runner-up.

6 Conclusion

In this paper, we illustrate a new black-box adversarial method, called OLIVANDER, that uses counterfactual explanations to identify potential vulnerabilities of an AI-based model (target model) trained for Windows PE malware detection using LIEF features. Specifically, OLIVANDER leverages counterfactual knowledge to generate adversarial Windows PE malware that can evade the target model. The experimental study explored the performance of two variants of the proposed evasion method obtained using either "padding" or "section injection" to manipulate a binary file. It also compared its performance to that of two black-box literature evasion methods—GAMMA and AMG. In addition, we analysed the transferability of the compared evasion methods against a state-of-the-art AI-based malware detection model and real-world anti-malware systems (accessed via VirusTotal). Finally, we also evaluate the offensive ability of the compared evasion methods against a decision model trained with an Adversarial Training approach.

Although this work starts exploring counterfactuals for black-box offensive adversarial learning, which is the most restrictive option, a limitation of this study is that it focuses on the application of counterfactual explanations within a specific feature engineering group (LIEF features) for Windows PE malware detection. As future work, we would explore how to establish the generalizability of the OLIVANDER method to other feature sets or domains. In addition, we would explore how the offensive performance of OLIVANDER may be possibly increased by using multiple file manipulations according to the idea evaluated by Kozák et al. (2024). We also intend to start investigating how counterfactuals can be leveraged for defensive adversarial learning as a means to mitigate the vulnerabilities of AI models for Windows PE malware detection to evasion methods. Finally, we intend to proceed with studying counterfactual explanations for Windows PE malware detection in a poisoning scenario.

Table 4 ♯Adversarial Malware and average Time (spent in minutes) of OLIVANDER^{PAD} and OLIVANDER^{INJ} by using DiCE-random and DiCE-gradient

Method	DiCE-random		DiCE-gradient	
	♯Adversarial Malware	Time (mins)	♯Adversarial Malware	Time (mins)
OLIVANDER^{PAD}	79	8.80	81	17.21
OLIVANDER^{INJ}	75	2.38	80	14.66

A Counterfactual algorithm performance analysis

In this Appendix we illustrate the results of a comparative study that was performed to examine the performance of DiCE-random and DiCE-gradient within OLIVANDER. DiCE-random is a model-agnostic method, while DiCE-gradient is a loss-based method (Mothilal et al. 2020) to generate counterfactuals. Table 4 reports the average Time spent in minutes completing the generation of each adversarial Windows PE malware and the number of adversarial Windows PE malware produced with OLIVANDER ^{PAD} and OLIVANDER^{INJ} using DiCE-random and DiCE-gradient. Both OLIVANDER^{PAD} and OLIVANDER^{INJ} were run in the default parameter configuration, i.e., with $\eta = 1000$ and $c = 100$. The results show that the use of DiCE-gradient slows the generation of adversarial Windows PE malware in both configurations of OLIVANDER (17.21 vs 8.80 min spent on average completing the elaboration of each Windows PE malware with OLIVANDER^{PAD}, and 14.66 vs 2.38 min spent on average completing the elaboration of each Windows PE malware with OLIVANDER^{INJ}). This mainly depends on the fact that DiCE-gradient spent more time than Dice-random to compute counterfactuals used to drive the generation of the adversarial counterparts of the considered Windows PE malware files. In fact, the DiCE-gradient spent 10.1 minutes on average generating the counterfactual of each Windows PE malware, while DiCE-random spent 1.93 min on average performing the same task. However, the more time spent generating counterfactuals with the DiCE-gradient leads to a slightly higher number of adversarial Windows PE malware that were produced thanks to the guidance of the gradient-based counterfactuals (79 vs 81 in OLIVANDER^{PAD}, and 75 vs 80 in OLIVANDER^{INJ}).

Acknowledgements Giuseppina Andresini is supported by the project FAIR - Future AI Research (PE00000013), Spoke 6 - Symbiotic AI, under the NRRP MUR program funded by the European Union - NextGenerationEU. Annalisa Appice and Donato Malerba are partially supported by project SERICS (PE00000014) under the NRRP MUR National Recovery and Resilience Plan funded by the European Union - NextGenerationEU. The research objectives of this paper are in partial fulfilment of the project AI-CREED (CUP H93C23000880005).

Author contributions LDR: conceptualization, methodology, software, data curation, validation, investigation, writing—review & editing. GA: conceptualization, methodology, investigation, visualization, supervision, writing—original draft, writing—review & editing. AA: conceptualization, methodology, investigation, project administration, funding, writing—original draft, writing—review & editing. DM: conceptualization, methodology, writing—review & editing.

Code availability https://github.com/Kyanji/OLIVANDER/.

References

Abel S, Tang Y, Singh J et al (2020) Applications of causal modeling in cybersecurity: an exploratory approach. ASTESJ 5(3):380–387. https://doi.org/10.25046/aj050349

Akhtar N, Mian A, Kardan N et al (2021) Advances in adversarial attacks and defenses in computer vision: a survey. IEEE Access 9:155161–155196. https://doi.org/10.1109/ACCESS.2021.3127960

Al-Essa M, Andresini G, Appice A et al (2024) PANACEA: a neural model ensemble for cyber-threat detection. Mach Learn 113(8):5379–5422. https://doi.org/10.1007/s10994-023-06470-2

Anderson HS, Roth P (2018) EMBER: an open dataset for training static PE malware machine learning models. CoRR abs/1804.04637:1–8. https://doi.org/10.48550/arXiv.1804.04637

Andriushchenko M, Croce F, Flammarion N et al (2020) Square attack: a query-efficient black-box adversarial attack via random search. In: ECCV 2020. Springer, pp 484–501. https://doi.org/10.1007/978-3-030-58592-1_29

Barut O, Zhang T, Luo Y et al (2023) A comprehensive study on efficient and accurate machine learning-based malicious PE detection. CCNC 2023:632–635. https://doi.org/10.1109/CCNC51644.2023.10060214

Brown A, Gupta M, Abdelsalam M (2024) Automated machine learning for deep learning based malware detection. Comput Secur 137:103582. https://doi.org/10.1016/j.cose.2023.103582

Chen H, Babar M (2024) Security for machine learning-based software systems: a survey of threats, practices, and challenges. ACM Comput Surv 56(6):1–38. https://doi.org/10.1145/3638531

Chen P, Zhang H, Sharma Y et al (2017) Zoo: zeroth order optimization based black-box attacks to deep neural networks without training substitute models. In: AISec'17, pp 15–26. https://doi.org/10.1145/3128572.3140448

Choras M, Wozniak M (2022) The double-edged sword of AI: ethical adversarial attacks to counter artificial intelligence for crime. AI Ethics 2(4):631–634. https://doi.org/10.1007/s43681-021-00113-9

Connors C, Sarkar D (2023) Machine learning for detecting malware in PE files. In: ICMLA 2023. IEEE, pp 2194–2199. https://doi.org/10.1109/COSMIC63293.2024.10871898

Cotae P, Reindorf N (2021) Using counterfactual regret minimization and Monte Carlo tree search for cybersecurity threats. BlackSeaCom 2021:1–6. https://doi.org/10.1109/BlackSeaCom52164.2021.9527857

Dahiya N, Chaudhary P (2021) PE file-based malware detection using machine learning. In: ICAIA 2020. Springer, pp 113–123. https://doi.org/10.1007/978-981-15-4992-2_12

Darias JM, Díaz-Agudo B, Recio-García JA (2021) A systematic review on model-agnostic XAI libraries. In: Workshops proceedings for the 29th international conference on case-based reasoning co-located with the 29th international conference on case-based reasoning (ICCBR 2021), Salamanca (Spain)/Online, September 13–16, 2021, CEUR workshop proceedings, vol 3017. CEUR-WS.org, pp 28–39

Demetrio L, Biggio B, Lagorio G et al (2021a) Functionality-preserving black-box optimization of adversarial Windows malware. IEEE Trans Inf Forensics Secur 16:3469–3478. https://doi.org/10.1109/TIFS.2021.3082330

Demetrio L, Coull SE, Biggio B et al (2021b) Adversarial EXEmples: a survey and experimental evaluation of practical attacks on machine learning for Windows malware detection. ACM Trans Privacy Secur 24(4):27:1-27:31. https://doi.org/10.1145/3473039

Emiris IZ, Fotakis D, Giannopoulos G et al (2024) GLANCE: global actions in a nutshell for counterfactual explainability. CoRR abs/2405.18921. https://doi.org/10.48550/ARXIV.2405.18921

Freiesleben T (2022) The intriguing relation between counterfactual explanations and adversarial examples. Minds Mach 32(1):77–109. https://doi.org/10.1007/s11023-021-09580-9

Goodfellow IJ, Shlens J, Szegedy C (2015) Explaining and harnessing adversarial examples. ICLR 2015:1–11
Gu J, Jia X, de Jorge P et al (2024) A survey on transferability of adversarial examples across deep neural networks. Trans Mach Learn Res 2024:1–35
Guidotti R (2022) Counterfactual explanations and how to find them: literature review and benchmarking. Data Min Knowl Discov. https://doi.org/10.1007/s10618-022-00831-6
Imran M, Appice A, Malerba D (2024) Evaluating realistic adversarial attacks against machine learning models for Windows PE malware detection. Future Internet 16(5):1–30. https://doi.org/10.3390/fi16050168
Khamaiseh SY, Bagagem D, Al-Alaj A et al (2022) Adversarial deep learning: a survey on adversarial attacks and defense mechanisms on image classification. IEEE Access 10:102266–102291. https://doi.org/10.1109/ACCESS.2022.3208131
Kozák M, Jurecek M, Stamp M et al (2024) Creating valid adversarial examples of malware. J Comput Virol Hack Tech. https://doi.org/10.1007/s11416-024-00516-2
Kreuk F, Barak A, Aviv-Reuven S et al (2018) Adversarial examples on discrete sequences for beating whole-binary malware detection. CoRR abs/1802.04528
Kuppa A, Le-Khac NA (2021) Adversarial XAI methods in cybersecurity. IEEE Trans Inf Forensics Secur 16:4924–4938. https://doi.org/10.1109/TIFS.2021.3117075
Li K, Guo W, Zhang F et al (2023) GAMBD: generating adversarial malware against malconv. Comput Secur 130:1–9. https://doi.org/10.1016/j.cose.2023.103279
Ling X, Wu L, Zhang J et al (2023) Adversarial attacks against Windows PE malware detection: a survey of the state-of-the-art. Comput Secur 128:1–24. https://doi.org/10.1016/j.cose.2023.103134
Liu FT, Ting KM, Zhou ZH (2008) Isolation forest. In: 2008 Eighth IEEE international conference on data mining, pp 413–422. https://doi.org/10.1109/ICDM.2008.17
Long T, Gao Q, Xu L et al (2022) A survey on adversarial attacks in computer vision: taxonomy, visualization and future directions. Comput Secur 121:102847. https://doi.org/10.1016/j.cose.2022.102847
Louthánová P, Kozák M, Jureček M et al (2024) A comparison of adversarial malware generators. J Comput Virol Hacking Tech. https://doi.org/10.1007/s11416-024-00519-z
Macas M, Wu C, Fuertes W (2024) Adversarial examples: a survey of attacks and defenses in deep learning-enabled cybersecurity systems. Expert Syst Appl 238:1–33. https://doi.org/10.1016/j.eswa.2023.122223
Madry A, Makelov A, Schmidt L et al (2018) Towards deep learning models resistant to adversarial attacks. ICLR 2018:1–10
Mothilal RK, Sharma A, Tan C (2020) Explaining machine learning classifiers through diverse counterfactual explanations. In: FAT 2020. ACM, pp 607–617. https://doi.org/10.1145/3351095.337285
Pawar JA, Avhankar MS, Gupta A et al (2024) Enhancing network security: leveraging isolation forest for malware detection. In: 2024 2nd International conference on advancement in computation & computer technologies (InCACCT), pp 230–234. https://doi.org/10.1109/InCACCT61598.2024.10550968
Pawelczyk M, Agarwal C, Joshi S et al (2022) Exploring counterfactual explanations through the lens of adversarial examples: a theoretical and empirical analysis. In: AISTATS 2022, vol 151. PMLR, pp 4574–4594
Pierazzi F, Pendlebury F, Cortellazzi J et al (2020) Intriguing properties of adversarial ML attacks in the problem space. In: SP 2020. IEEE, pp 1332–1349. https://doi.org/10.1109/SP40000.2020.00073
Raff E, Barker J, Sylvester J et al (2018) Malware detection by eating a whole EXE. In: AAAI 2018 workshops, pp 1–8
Shi T, McCann RA, Huang Y et al (2024) Malware detection for internet of things using one-class classification. Sensors. https://doi.org/10.3390/s24134122
Szegedy C, Zaremba W, Sutskever I et al (2014) Intriguing properties of neural networks. ICLR 2014:1–10
Şandor M, Portase RM, Coleşa A (2023) Ember feature dataset analysis for malware detection. ICCP 2023:203–210. https://doi.org/10.1109/ICCP60212.2023.10398693
Thomas R (2017) Lief—library to instrument executable formats. https://lief.quarkslab.com/
Wang M, Wang D, Wu W et al (2023) T-COL: generating counterfactual explanations for general user preferences on variable machine learning systems. CoRR abs/2309.16146. https://doi.org/10.48550/ARXIV.2309.16146
Yuan Y, McAreavey K, Li S et al (2024) Multi-granular evaluation of diverse counterfactual explanations. In: ICAART2024, vol 2. SciTePress, pp 186–197. https://doi.org/10.5220/0012349900003636

Zhiyi T, Lei C, Jie L et al (2022) A comprehensive survey on poisoning attacks and countermeasures in machine learning. ACM Comput Surv 55(8):1–35. https://doi.org/10.1145/3551636

Authors and Affiliations

Luca De Rose[1] · Giuseppina Andresini[1,2] · Annalisa Appice[1,2] · Donato Malerba[1,2]

✉ Luca De Rose
luca.derose@uniba.it

Giuseppina Andresini
giuseppina.andresini@uniba.it

Annalisa Appice
annalisa.appice@uniba.it

Donato Malerba
donato.malerba@uniba.it

[1] Department of Computer Science, Università degli Studi di Bari Aldo Moro, Via Orabona, 4, 70125 Bari, Italy

[2] Consorzio Interuniversitario Nazionale per l'Informatica, Bari, Italy